

Apr'26 Major Examination: EEL71020 Hardware Design for AI (MTECH-AI Batch)

Guidelines (Total time: 180 minutes, Maximum Marks: 52):

- Please read the question paper very carefully. **NO clarification is required in any question.** In case of any doubt, assume whatever you wish to and state that in your answer.
- Usage of LLM/AI tool is NOT allowed. NOTES can be referred to.

1. Transformer models have received widespread attention in recent years. They have gradually replaced recurrent neural networks (RNNs) in natural language processing (NLP) and are widely used in tasks such as machine translation, text generation, and language understanding. Similarly, transformers have shown impressive results in computer vision (CV). However, their unique attention mechanism places high demands on the computational and storage resources of the hardware. Basics of transformer model is shown in below figure:

- Query, Key, Value projections using matrix multiplication:

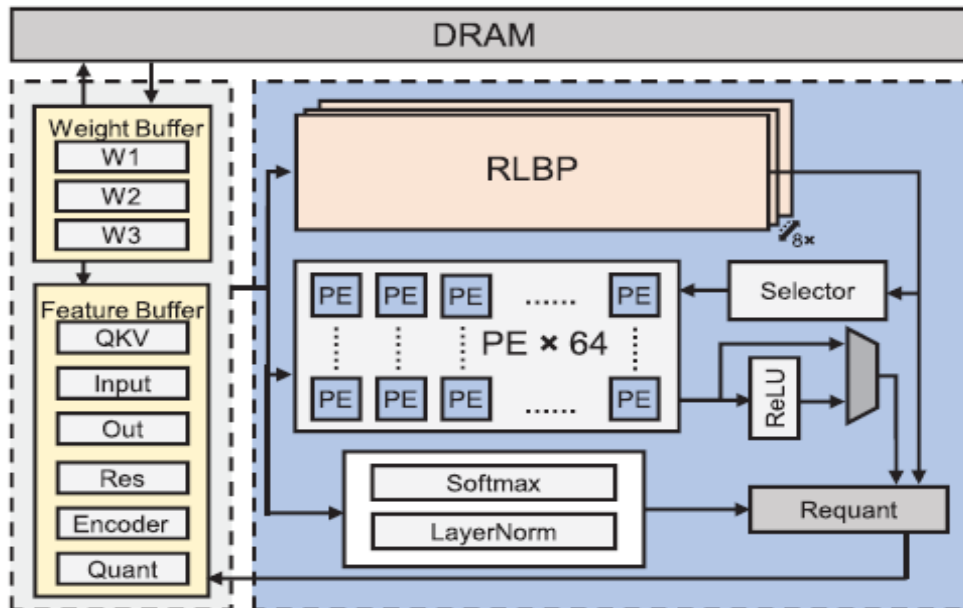
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

- Attention output:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

Deploying transformers on edge computing platforms is challenging due to their complex data flow, intensive matrix calculations, and the need for high-precision nonlinear functions. To address these challenges, a new type of accelerator micro-architecture is needed. Below is a scheme for designing reconfigurable transformer accelerator (RTA) with four major innovations:

- **Dynamic Sparsity in Transformer-Based Models:** We employ a low-bit-width estimation method to induce dynamic sparsity in the attention and FFN computations of transformer-based models. With acceptable accuracy degradation, we achieve a filtering rate of 60%–80% for QK^T in attention operation and 70%–90% for FFN1 in the FFN operation. The proposed scheme for this accelerator design is shown as below:



- **Decomposition Method for Constructing Large-Bit- Width Multipliers Using Low-Bit-Width Multipliers:** We propose a decomposition method to construct a large bit- width multiplier using low-bit-width multipliers. Specifically, we build an 8-bit signed multiplier using eight 4-bit signed multipliers, along with simple shift and addition/subtraction operations.
- **Reconfigurable Low-Bit-Width Prediction (RLBP) Module:** Combining the efforts from 1) and 2), we design the RLBP module using reconfigurable technology. The RLBP module can perform both low-bit-width prediction and large-bit-width dot product computations. Compared to adding a separate 64-D 8-bit vector dot product module on the basis of the filtering module, the RLBP module reduces lookup table (LUT) consumption by 11.5% and FF by 93.9%.
- **Complete Transformer Hardware Accelerator RTA:** We design the complete transformer hardware accelerator RTA, which leverages feature sharing, ping-pong buffering, and other techniques to minimize resource consumption while maximizing performance. Compared to the current state-of-the-art transformer hardware accelerators, RTA achieves a digital signal processor (DSP) efficiency improvement of 1.37–11.03x.

Answer following questions with the above scenario:

- (a) In one attention head, let sequence length be $N=128$ and key dimension be $d_k = 64$. suppose the low-bit-width estimator filters out 75% of the total QK^T operations (which are essentially 8-bit MAC operations). The PE array contains 64 PEs, and each PE performs 1 MAC per cycle at 250 MHz. Calculate the total number of cycles after the sparsity?
- (b) A transformer accelerator processes a batch of 4 sequences, each of length $N=64$. Assume: $d_{model}=512$ and $d_{ff}=2048$, If dynamic sparsity filters out 87.5% of FFN1 operations, and the same 64-PE engine runs at 200 MHz. Compute the time (in ms) **Rounded-off to two places of decimal?**
- (c) To keep the compute engine stall-free, the next tile must be loaded while the current tile is being processed. For one tile: Feature tile read = 32 KiB, Weight tile read = 24 KiB, and Output tile write-back = 32 KiB

It is known that total data movement per tile happens within the compute time of $30 \mu s$. Given the above information, the minimum DRAM bandwidth (in GB/s) needed to fully hide memory transfer is 1.86 GB/s. Is this statement true or false?

- (d) Suppose in a baseline design, the Q, K, and V features are fetched separately from DRAM: Q read = 64 KiB, K read = 64 KiB, and V read = 64 KiB

We devise a new scheme: with feature sharing, **only one 64 KiB read is needed.**

Assume DRAM bandwidth is 8 GB/s, and compute time for that stage is fixed at $40 \mu s$ with no overlap. Find total new stage latency (in μs) **Rounded-off to two places of decimal?**

- (e) Consider one encoder block with: $N=128$, $d_{model} = 512$, $h = 8$ heads, $d_k = 64$, and $d_{ff} = 2048$. All remaining dense operations (QKV projection, AV, FFN2, etc.) together cost 50,331,648 MACs. Given the filtering estimates as mentioned in part-(a) and part-(b) and the information that the PE array has 64 PEs running at 250 MHz. Find out the sparse latency (in ms) **Rounded-off to two places of decimal?**

- (f) An RTA tile performs both sparse QK^T and sparse FFN1 for one token block.

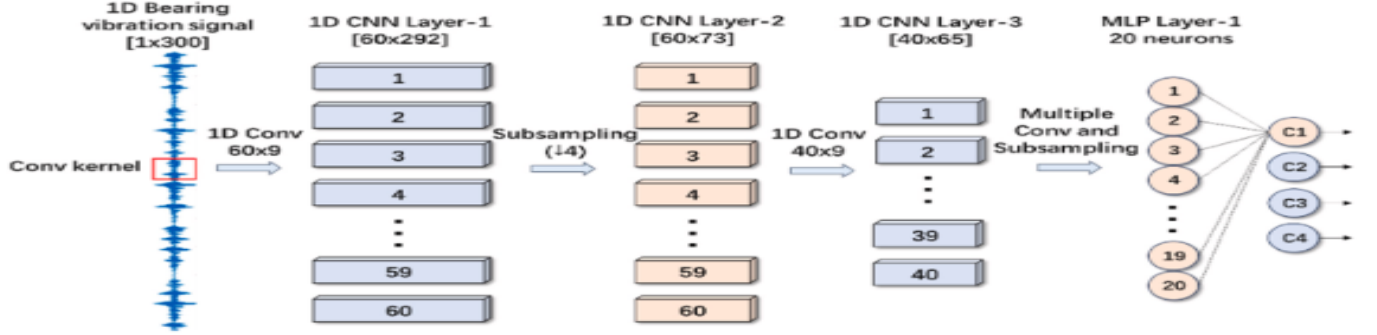
Assume per tile:

Sparse QK^T work = 262,144 MACs, Sparse FFN1 work = 524,288 MACs, and Other dense work = 131,072 MACs

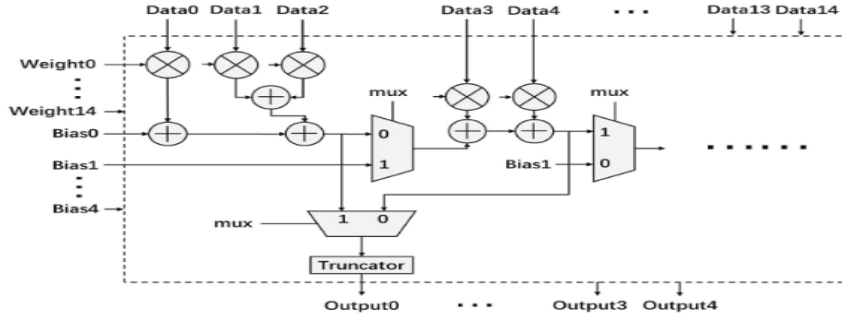
The tile is executed on a 64-PE engine at 400 MHz. Further assume Softmax, LayerNorm, and Requant add a fixed 320 cycles per tile. Under this scenario, check if the system throughput is 24594.38 tiles/s.

2. Compared to classical 2D convolutional neural networks (2D-CNNs), one-dimensional convolutional neural networks (1D-CNNs) offer several advantages, including lower computational complexity, a more compact

structure, and natural suitability for processing one-dimensional data signals. Consequently, 1D-CNNs are extensively employed across various domains, such as arrhythmia detection, automatic speech recognition, and structural damage detection. Below Figure lustrates the schematic diagram of a compact 1D-CNN architecture that is applied for damage detection in bearing structures.



In the design of accelerator for the above workload, a custom block involving a multiplier-adder tree array (as shown below where each multiplier is 16 bits and each adder is 32 bits) is designed. The multiplier-adder tree array consists of 15 multipliers and 15 adders, which can be configured into multiplier-adder tree of size 3 or 5 using a multiway selector. The final operation result is truncated to 16 bits using a truncator. Convolution operations with kernel sizes less than or equal to 5 can be completed in a single cycle, while those with kernel sizes greater than 5 are completed over multiple cycles using intermediate data registers.



For convolution layers with kernel sizes smaller than 15, parallel acceleration across multiple convolution kernels is possible. For layers with kernel sizes greater than or equal to 15, acceleration is achieved for a single convolution kernel computation. The comparator array is composed of 10 comparators and 10 registers, supporting up to 10 parallel comparison operations. This array can be configured to perform ReLU functions as well as maximum pooling operations. When the mux signal is high (meaning as "1"), all mux devices route the "1" path open and the "0" path closed, resulting in $\text{Output0} = \text{Data0Weight0} + \text{Data1Weight1} + \text{Data2Weight2} + \text{Bias0}$, with similar outputs for other Output ports. At this time, the multiplier-adder tree array is configured as five filters array of size 3. Conversely, when the mux signal is low (meaning as "0"), $\text{Output0} = \text{Data0Weight0} + \text{Data1Weight1} + \text{Data2Weight2} + \text{Data3Weight3} + \text{Data4Weight4} + \text{Bias0}$, with similar outputs for Output1 and Output2 ports, while Output3 and Output4 ports produce meaningless outputs, and bias3 and bias4 ports are unused. During this configuration, the multiplier-adder tree array is arranged as three filters array of size 5.

- (a) A 1D input signal of length 300 is processed by 60 valid 1D convolution filters of length 9, producing the first layer output of size 60×292 . The custom multiplier-adder tree is configured in size-5 mode, so it computes 3 filters in parallel, and each 9-tap convolution output requires 2 cycles: first cycle for 5 taps, second cycle for the remaining 4 taps plus accumulation with the intermediate partial sum. ReLU is applied after all convolutions using the comparator array, which can process 10 values per cycle. The clock frequency is 250 MHz. Please find the total latency of Layer-1 in microseconds (**Rounded to the nearest integer**)?

- (b) Consider that the Layer-1 with 60 filters of length 9 executed in size-5 mode. In cycle 1, the accelerator reads 5 input samples and 15 weights; in cycle 2, it reads 4 input samples and 12 weights. Assume all inputs and weights are 16 bits, each output is written as 16 bits, and biases are preloaded and do not contribute to per-position traffic. Ignore address/control overhead. Now, consider the following statements:
 (i) Peak read bandwidth is 320 bits/cycle, (ii) Average read bandwidth over the 2-cycle schedule is 260 bits/cycle (iii) Total number of output writes for the full layer (i.e., layer-1) is 17520 writes. Consider following options and identify the correct out of them: **A.** All are True, **B.** None is True, **C.** (i) & (ii) are True, **D.** (i) & (iii) are True
- (c) Consider the scenario of (b) as mentioned above. It is known that the weights of each 3-filter group are loaded once and reused across all 292 positions. Compute the traffic-reduction factor due to on-chip weight reuse?
- (d) Assume a modified version of the network where an input tensor of size 60×73 is processed by 40 filters of length 15 with valid convolution and stride 1. Because the kernel size is 15, the architecture can accelerate only one filter at a time, but each output sample can be completed in one cycle using the full 15-multiplier tree. After convolution, ReLU is applied, followed by non-overlapping max-pooling of size 2 and stride 2. The comparator array supports 10 parallel comparison operations, and a size-2 max-pool requires 1 comparison per pooled output. Use a 200 MHz clock. Please find the total end-to-end latency of this layer block in microseconds (**Rounded to the nearest integer**)?
- (e) Fixed-point number presentations are frequently used in the representation of weights of neural networks. For example, Q1.15 is a fixed-point signed number format with: 1 sign/integer bit and 15 fractional bits. So, the total word length is 16 bits. Range example is illustrated as below:

Since it is signed 16-bit:

- minimum integer value = -32768
- maximum integer value = 32767

Therefore real-value range is:

$$\frac{-32768}{2^{15}} = -1.0$$

$$\frac{32767}{2^{15}} \approx 0.9999695$$

So the range is:

$$\boxed{-1.0 \text{ to } 0.9999695}$$

If the stored 16-bit signed integer is:

$$16384$$

then the real value is:

$$\frac{16384}{32768} = 0.5$$

If the stored value is:

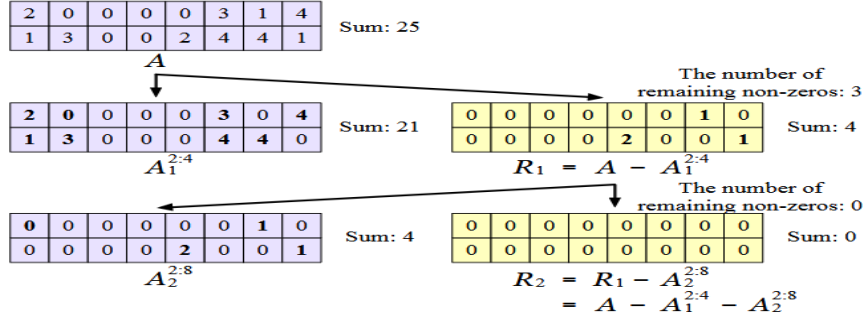
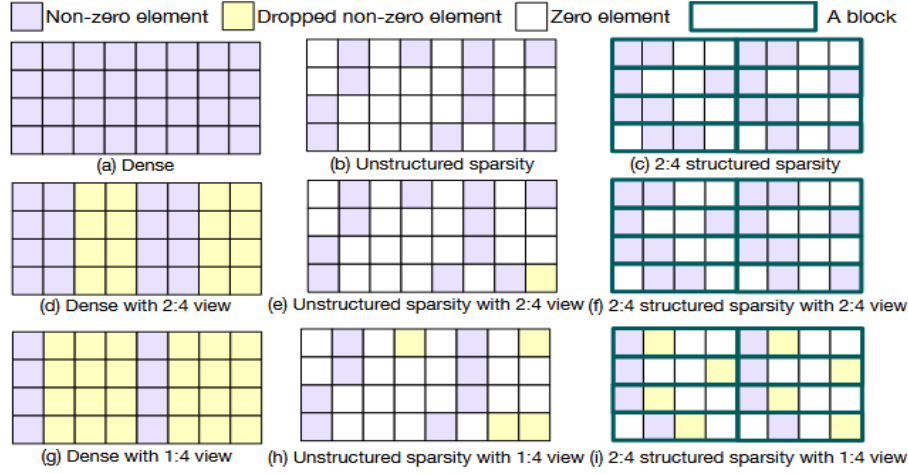
$$-8192$$

then the real value is:

$$\frac{-8192}{32768} = -0.25$$

Assume each input sample and each weight is represented in signed Q1.15 format. Therefore, each product is formed in full precision before accumulation. A 9-tap convolution output is computed as the sum of 9 such products plus a signed bias in Q5.27 format. The hardware uses 32-bit adders and then truncates the final result to 16 bits. Given this information, Check if a **signed 32-bit accumulator is sufficient after aligning the bias to the product fractional format.**

3. Exploiting sparsity in deep neural networks (DNNs) has been a promising area to meet the growing computation needs of modern DNNs for a range of deployment scenarios. However, in practice, sparse DNN acceleration still faces a key challenge. To minimize the overhead of sparse acceleration, hardware designers have proposed structured sparse hardware support (corresponding to different types of sparsity patterns as shown in below Figure), which provides limited flexibility and requires extra model fine-tuning. Any sparse model fine-tuned for certain structured sparse hardware cannot be accelerated by other structured hardware. So, to bridge gap between sparse DNN models and hardware, one idea is to have tensor approximation via structured decomposition (TASD), which leverages distributive property in linear algebra to turn any sparse tensor into a series of structured sparse tensors (as outlined in the below Figure).



A view of a tensor A for a sparsity pattern is a tensor after potentially dropping some elements to meet the rule of the sparsity pattern. For a matrix filled with non-zeros randomly, it is possible that the matrix does not meet the 2:4 sparsity pattern, i.e., there could be a block composed of 4 consecutive elements with more than two non-zeros. To generate a 2:4 view of the matrix, some non-zeros in the matrix should be dropped (pruned in DNNs) to meet the pattern. It can be clearly understood from the above figure that the tensor A is equal to the sum of $A_1^{2:4}$ and $A_2^{2:8}$. Therefore, this decomposition can be generalized for any dimension with the help of distributive property. TASD decomposes any tensor A into a series of structured sparse tensors. The decomposed tensors can be used in any tensor algebra, such as matrix multiplication ($C = A \times B$), which can be approximated as A_{1B}^{s1} , so if $s1$ is 2:4, and the matrix multiplication is running on NVIDIA Sparse Tensor Core (STC), potentially 50% of Multiply-and-Accumulate (MAC) operations could be skipped. However, this percentage of reduction turns out to be a strong function of the attempted tensor decomposition. Thus, finding the proper TASD series to minimize the error while maximizing compute reduction determines the quality of the approximation.

- (a) A 64×64 systolic array performs matrix multiplication $C = A \times B$, where: A is 512×512 , B is 512×512 . For dense execution: One PE performs one MAC per cycle, Total cycle count for one tiled execution is: $K + (M_t - 1) + (N_t - 1)$ for each $M_t \times N_t$ with inner dimension K .

Now suppose the weight matrix is converted into 2:4 structured sparsity, and the hardware can skip exactly half of multiplications without pipeline stalls. Assume the following: Tile size = 64×64 , $K = 512$ for every tile, There are no DRAM stalls, Dense and sparse executions use the same fill and drain overhead, and Only the multiplication phase is reduced by sparsity whereas fill+drain remain unchanged. Let the final speed-up with above decomposition in this scenario be S . What is $3 \times S$?

- (b) A transformer feed-forward layer performs two GEMMs:
GEMM-1: 4096×1024 multiplied by 1024×4096 GEMM-2: 4096×4096 multiplied by 4096×1024

Three execution modes are available as shown in the below figure:

Assume the same peak rate, 300 TMAC/s, applies to all three modes, and the base MAC energy is 0.42 pJ per executed MAC. Energy-delay product (EDP) is a very important metric in deciding overall system efficiency. It is calculated as product of latency (i.e., runtime) and the energy con-

Mode A: Dense	Mode B: Native 2:4	Mode C:TASD (PROPOSED ABOVE)
1. MAC energy = 0.42 pJ 2. Runtime = proportional to total MACs at 300 TMAC/s	1. Only 60% of weights can be legally converted to 2:4 2. Those converted weights enjoy 50% MAC skipping 3. The remaining 40% run dense 4. Sparse control overhead adds 8% extra energy over total compute energy	1. 90% of all original weights are represented through structured views: • 65% through 2:4 views • 25% through 2:8 views 2. 10% are dropped 3. 2:8 gives 75% MAC skipping 4. Metadata/control overhead adds 15% extra energy 5. Same peak engine rate: 300 TMAC/s 6. Ignore decomposition time

sumed. **Please calculate the EDP for Mode A (dense mode) in the unit of $\mu J \cdot seconds$ (Round off to the nearest integer)?**

- (c) Consider the scenario as shown above, Compute the value of EDP metric for Mode-C (TASD mode)?
- (d) An NPU has the following on-chip memories: Input SRAM = 192 KB, Weight SRAM = 256 KB, and Output SRAM = 128 KB

It executes a convolution layer: Input feature map = $112 \times 112 \times 32$, Output channels = 96, Kernel = 5×5 , Stride = 1 and Output spatial size = 108×108 .

The supported data precision: Activations = 8 bits, Weights = 8 bits, and Partial sums = 24 bits. The compiler chooses a tile in this manner: Input tile spatial size = $T_h \times T_w = 20 \times 20$, and Output channel tile = $T_c = 24$. Further, assume the following specifications:

- For a 5×5 kernel and stride 1, required input halo is already included in the 20×20 tile
- Double buffering is used for input and weights only
- Output tile is not double-buffered
- One MAC array has 1536 MAC units
- Clock = 600 MHz
- MAC utilization due to boundary effects = 82%
- Ignore DMA (Direct Memory Access) overlap penalties

Please compute the final estimated latency (in milliseconds i.e., ms) Round off to the nearest integer?

- (e) Consider again the problem as shown in (d) above for NPU-based deployment. Compute the output SRAM capacity needed per tile?